

QuizArena

Full Stack Assessment -- Final Assignment Document

React / TypeScript | Node.js / Express | MongoDB | JWT

QuizArena is a multi-role quiz platform with three personas – Admin, Instructor, and Participant/User (Candidate).

This document defines the complete full-stack scope across seven milestones covering both frontend (React/TypeScript) and backend (Node.js/Express) deliverables. Build milestones in order -- every later milestone assumes the earlier ones are correct.

Each milestone is split into FRONTEND REQUIREMENTS (screens, UI behaviour, RTK Query / Redux integration) and BACKEND REQUIREMENTS (API endpoints, rules, constraints). Anything under RULES & CONSTRAINTS is non-negotiable.

Prerequisites

Frontend Stack

- **TypeScript only** -- no any anywhere in the codebase.
- All APIs via **RTK Query**.
- **Atomic Design** structure: atoms, molecules, organisms.
- **SCSS only** for styling; no external CSS libraries.
- Clean folder structure: **components, utils, types, interfaces, assets, pages**.

Backend Recommended Stack

- **Node.js 18+**, TypeScript, Express
- **MongoDB**, Mongoose
- **JWT + HTTP-only Cookie** (cookie-parser)
- **bcrypt** for password hashing
- **Zod / Joi** for input validation; **multer** for CSV uploads
- **Jest + Supertest** for testing
- **Swagger UI / OpenAPI** for API documentation

Milestone Roadmap

M1	Authentication Flow	Login, Signup, Post-Login UI Auth API, JWT, Middleware
M2	Admin Flow	Admin Dashboard, Quiz & Access Mgmt UI User/Quiz APIs, RBAC
M3	Instructor Flow	Instructor Dashboard, Quiz Authoring UI Questions, Participants, Publish
M4	User / Candidate Dashboard	User Dashboard, Upcoming Quizzes UI Quiz Discovery, Assignment Gu
M5	Quiz Attempt Flow	Test Screen, Timer, Submission UI Examination Engine, Scoring, Integri
M6	Results & Analytics Flow	Result Screens (all 3 roles) Per-Role Result APIs, Determinism
M7	Brownie Points	Dark Theme, Responsive, FE Tests >90% BE Tests >=80%, Docs, Oper

Milestone 1: Authentication Flow

Covers the complete authentication journey and lays the secure backend foundation every later milestone depends on. New users sign up (defaulting to Participant/User role); existing users log in. Backend stores passwords via bcrypt, issues a signed JWT into an HTTP-only cookie, and protects every authenticated route with a single requireAuth middleware.

FRONTEND REQUIREMENTS

Login Screen

- Login screen developed as per design with correct layout, spacing, colours, and typography.
- Proper error handling for **wrong credentials**, **empty fields**, and **network errors** with inline error messages.
- Login API integrated via **RTK Query**; login state managed through **Redux slices**.
- On successful login, **JWT token** received and stored securely (preferably in **HTTP cookies**).

Sign Up Screen

- Sign-up screen developed as per design with correct form layout, inputs, button, and validation states.
- Error handling for **required fields**, **invalid email**, **weak password**, and **duplicate email**.
- Sign-up API integrated via **RTK Query**; sign-up state managed through **Redux slices**.
- After successful sign-up, show success toast '**Signup successful**' and redirect to Login screen.

Post-Login Flow

- After successful login, redirect user to the **dashboard**.
- Navbar implemented as per design; **live time** shown in navbar updating in real time with **seconds accuracy**.
- Logout: delete **JWT token** from browser and **clear Redux state**.

BACKEND REQUIREMENTS

1.A Authentication API

User Registration [POST /auth/signup]

- Accept name, email, password (role defaults to **candidate**).
- Hash password with **bcrypt** before persisting -- **never store plain text**.
- Reject duplicate emails with **HTTP 409 Conflict**; malformed input with **400**.
- Respond with the created user (**without the password hash**) and a success status.

User Login [POST /auth/login]

- Validate credentials against the **bcrypt** hash.
- Issue a signed **JWT** as an **HTTP-only, Secure, SameSite** cookie -- **never** return the token in the body.
- Return user profile (no password). Use **HTTP 401** for bad credentials; **do not reveal** whether the email exists.

User Logout [POST /auth/logout]

- Clear the **JWT cookie** cleanly (same name, path, expiry in the past).
- **Idempotent** -- calling logout when already logged out must still succeed.

Get User Profile [GET /auth/me]

- Read current user from the **JWT cookie** and return their profile.
- **Never** include the password field -- strip it at the model layer.
- Return **401** when the cookie is missing or invalid.

1.B Middleware & Project Setup

Route Protection Middleware

- Single **requireAuth** middleware: parses cookie, verifies JWT, attaches **req.user**.
- Reject unauthenticated requests with **401**; tampered/expired tokens with a clear error.
- Apply to **every protected route** -- no leaky endpoints.
- Centralize error responses through a single error-handling middleware.

RULES & CONSTRAINTS

- Passwords **always** hashed with **bcrypt (cost >= 10)**.
- JWT lives **only** in an **HTTP-only cookie** -- never in localStorage, never in the response body.
- Profile responses must **never** carry the password hash, even by accident.
- Use **environment variables** for JWT secret, cookie domain, and bcrypt cost.

Milestone 2: Admin Flow

The Admin owns the platform -- managing users, creating quizzes, assigning instructors, and policing role boundaries. Frontend covers Admin Dashboard, Create/Edit Quiz, and Access Management screens. Every backend endpoint must be locked behind an admin-only guard.

FRONTEND REQUIREMENTS

- **Admin Dashboard** developed with **sidebar**, **header**, **stat cards**, **donut/pie chart**, and **quizzes table**.
- Pie chart shows **Completed**, **Scheduled**, **Draft**, **Cancelled** with legends.
- Table includes correct columns, **status colors**, actions, **pagination**, and exact **date format**.
- **Search** functionality on the quizzes table.
- Admin can **create a quiz** with fields/labels/date picker/question type toggle; mandatory validations for title/date/time/duration/question type.
- Admin can **edit quiz details** -- auto-populated data + update validation.
- Admin can **cancel a quiz** with a confirmation pop-up.
- Admin manages **Instructor access**: Assign Instructor by email + error handling for already-instructor / email not found.
- Quiz state restriction: **Admin cannot delete a Live quiz**.
- Two empty states: **no quizzes** and **no search results**.

BACKEND REQUIREMENTS

2.A User Management API

List & Filter Users [GET /admin/users]

- Pagination (**page**, **limit**) + filtering by **role** and **status**.
- Include **user-count statistics** in the same response -- avoid a second API call.
- Support sorting and search by **name/email**. **Never** return password hashes.

2.B Quiz Management API

Create Quiz [POST /admin/quizzes]

- Accept metadata (title, description, schedule window, duration). Initial status: **Draft**.
- All datetimes as **ISO-8601 (UTC)**. **endTime** must be strictly after **startTime**.
- Reject malformed payloads with **400** and a clear field-level error message.

Assign Instructor [POST /admin/quizzes/:id/assign-instructor]

- Validate target user exists and holds the **instructor role**.
- Return meaningful **4xx error** if user does not exist or is not an instructor.

Edit Quiz **[PATCH]** /admin/quizzes/:id]

- Allow edits **only** while quiz is **Draft** or **Scheduled**.
- Reject edits to **Live**, **Completed**, or **Cancelled** quizzes.
- Validate state transitions: **Draft -> Cancelled** is fine; **Cancelled -> Live** is not.

Delete Quiz **[DELETE]** /admin/quizzes/:id]

- **Only Draft** quizzes can be deleted.
- Scheduled / Live / Completed must be **cancelled**, not deleted.
- Return **409 Conflict** when the state guard blocks the action.

Cancel Quiz **[POST]** /admin/quizzes/:id/cancel]

- Flip status to **Cancelled**; record a **cancellation timestamp**.
- Once cancelled, the quiz is **read-only** -- no further edits, no submissions.
- **Completed** quizzes **cannot** be retroactively cancelled.

2.C Access Control

Role-Based Access Control

- **requireRole('admin')** guard on every **/admin/*** route.
- Non-admin roles hitting admin routes receive **403 Forbidden** (not 401).
- Role check **always runs after** the auth check -- never trust client-supplied role headers.

RULES & CONSTRAINTS

- Quiz lifecycle: **Draft -> Scheduled -> Live -> Completed**; **Cancelled** reachable from Draft/Scheduled/Live.
- All datetimes **ISO-8601**, stored in **UTC**.
- Stats and lists **ship together** when the UI shows them together.

Milestone 3: Instructor Flow

Instructors manage quizzes assigned to them -- from Draft through to Published. Frontend covers Instructor Dashboard, Update/Edit Quiz screen, Questions tab, and Participants tab. Every backend action is scoped to quizzes the instructor owns; no instructor may touch another's quiz.

FRONTEND REQUIREMENTS

Instructor Dashboard

- **Pixel-perfect** Instructor Dashboard with **donut/pie chart** (Completed / Scheduled / Cancelled with legends).
- **My Quizzes table** with correct columns, status colour-coding, action column, date format (**DD MMM YYYY . HH:MM AM/PM**), and pagination.
- Action column: **Edit** for Draft/Scheduled, **View Result** for Completed, -- for Cancelled.
- Empty-state UI when **no quizzes exist** and when **search returns no results**.
- Dashboard data via **single API + Redux**; search with **debounce/throttling**.

Update Quizzes Screen

- Shows **only Draft and Scheduled** quizzes.
- Table includes **Readiness status**, working **Edit** button, correct date format, and pagination.
- Draft and Scheduled quizzes fetched via **single API + Redux + pagination**.
- Search via **API + Redux + debounce/throttling**.

Edit Quiz Screen

- Edit Quiz screen with top bar, tabs, schedule section, and questions list.
- Working **breadcrumbs**, **Back to Quiz List**, **Cancel Quiz**, and **Publish/Re-publish** buttons.
- Date/time selection via picker controls; preview shown correctly; **API integration through Redux**.
- Quiz schedule banner shown based on quiz state with **proper colour coding**, updates **without page reload**.

Questions Tab

- **Add via CSV** and **Add Manually** actions.
- Manual add: modal with question text, options, correct answer; mandatory field validations; data refresh **without page reload**.
- CSV bulk upload: modal + **frontend validations before enabling import + API + Redux**.
- Existing questions listed with **Edit** and **Delete** actions; Edit opens prefilled modal; Delete includes confirmation.

Participants Tab

- **Add via CSV** and **Add Manually** (by email) actions; list with **Remove** button.
- Manual add validates that email belongs to a **registered user**.
- Participants removable **only before the quiz goes Live**; removal includes confirmation + data refresh.
- **Show More / pagination** for both questions and participants lists (backend pagination + Redux).

Publish / Re-publish

- **Draft readiness** clearly indicates whether the quiz is ready to publish.
- Quiz publishable **only when at least one question exists** and **at least one participant** is assigned.
- **Publish** button calls publish API via Redux; **Re-publish** available after editing a Scheduled quiz.
- Scheduled quiz data **auto-populated** in edit mode with proper error handling.
- Cancel Quiz flow **reuses the same API, modal, and components** from Milestone 2.
- Proper **empty-state screens** for: no quizzes, no search results, no questions, no participants.

BACKEND REQUIREMENTS

3.A Quiz Management API

List Quizzes (Instructor) [GET /instructor/quizzes]

- Return **only quizzes assigned** to the authenticated instructor.
- Support sort, filter (by status), and pagination; include **readiness signals**.

Edit Quiz (Instructor) [PATCH /instructor/quizzes/:id]

- Allow edits **only on Draft or Scheduled** quizzes.
- **Ownership guard** -- return **403** if quiz is not assigned to this instructor.

Question Management

- Manual (one at a time) and **CSV bulk upload**.
- **Validate every CSV row** before insertion -- reject the **whole upload** on errors with row-level messages.
- Supports: **single-choice**, **multi-select**, **true/false**.

Participant Management

- Add by email or via CSV; validate user exists and holds the **candidate role**.
- Record **addedAt timestamp** per participant. Removal allowed **before quiz goes Live**.

Manage Questions APIs [POST . PATCH . DELETE /instructor/quizzes/:id/questions]

- Full CRUD scoped to owning instructor; CSV bulk-insert included.
- Allow question edit **only in Draft State**.

Manage Participants APIs [POST . DELETE /instructor/quizzes/:id/participants]

- Add/remove for Draft/Scheduled quizzes; reject **non-candidate** role with clear message.
- **Idempotent** -- re-adding an existing participant must **not duplicate** the entry.

Publish Quiz [POST /instructor/quizzes/:id/publish]

- Transition: **Draft -> Scheduled**.
- Pre-conditions: **at least 1 question, at least 1 participant**, valid schedule window.
- Return **409 Conflict** with list of missing readiness items when pre-conditions fail.

Cancel Quiz (Instructor) [POST /instructor/quizzes/:id/cancel]

- **Terminal** -- no resurrecting a cancelled quiz. Ownership enforced.

Delete Quiz (Instructor) [DELETE /instructor/quizzes/:id]

- **Draft only** and owned by this instructor; use cancel for any later state.

3.B Access Control -- Ownership Rules

- Every route verifies quiz is assigned to current user; **403** (not 404) for others' quizzes.
- Centralize the ownership check in a **reusable guard** -- never duplicate per controller.

RULES & CONSTRAINTS

- CSV uploads are **atomic**: all rows pass and insert, or **nothing inserts**.
- **Publish must enforce readiness** -- never let a quiz with zero questions or participants go live.
- State guards apply **equally** to instructors.

Milestone 4: User / Candidate Dashboard and Upcoming Quiz Flow

Participants (Candidates) see only quizzes assigned to them. Quizzes appear in the appropriate state-based section: Upcoming, Live, Completed, or Expired. Backend strictly enforces assignment-only visibility; questions and answers are never exposed from discovery endpoints.

FRONTEND REQUIREMENTS

- **User Dashboard** with **stat cards + quizzes table**.
- Table actions: **Start Test** for Live | **View Result** for Completed | **date** for Upcoming | -- for Cancelled.
- **Single API + Redux + pagination** for the table; search with **debounce/throttling**.
- **Upcoming quiz cards**: color coding, correct date, **real-time countdown**, disabled **Start** button until Live.
- **Assigned-only visibility** enforced -- user never sees a quiz not assigned to them.
- User can view results of attempted quizzes in **detailed form**.
- Both empty states: **no quizzes initially** and **search returns no results**.

BACKEND REQUIREMENTS

4.A Quiz Discovery

Get Quizzes (Candidate) [GET /candidate/quizzes]

- Return **only quizzes** the candidate is a **participant in**.
- Filter (upcoming/live/completed), sort, pagination; bundle **per-status counts** in same response.
- Strip instructor-private fields -- **no question payload, no answer keys**.

Quiz Details (Pre-Attempt) [GET /candidate/quizzes/:id]

- Return metadata **only**: title, description, duration, schedule, question count.
- **Never expose** questions or answer options.
- Apply a **live-window guard** -- joinable details only inside the scheduled window.

4.B Access Control -- Assignment Visibility

- **403 or 404** for any quiz the candidate is not assigned to.
- Pagination/filtering happens **after** the assignment filter -- never leak total counts of unassigned quizzes.
- Direct-URL probing should **never** reveal the existence of unassigned quizzes.

RULES & CONSTRAINTS

- Candidates **never** see questions or answers from a discovery endpoint -- gated by M5 start endpoint.
- **Live-window guard** must be applied on pre-attempt details.
- All candidate routes scoped **strictly** to the assignment set -- **no exceptions**.

Milestone 5: Quiz Attempt Flow / Examination Engine

The quiz-taking journey. Frontend: enter only during Live window, navigate questions, retain answers, manual submit or auto-submit on expiry. Backend: the start endpoint is the only place questions leave the server (correct answers stripped); submit commits exactly once per candidate.

FRONTEND REQUIREMENTS

- Quiz start flow implemented: **Pre-test modal, Test Screen, Post-Submit screen** matching design.
- **Question navigator** with status colors: **visited / visited+answered / not-visited**.
- Entry only during the **Live window**; **Live-window URL access control** blocks access outside it.
- **Timer starts on quiz open** (not on scheduled start time).
- **Manual submit** includes a confirmation step; **auto-submit** occurs when timer reaches **zero**.
- Navigation between questions works freely; answers **persist** when revisiting questions.
- After submission: **session lock** -- participant cannot re-enter.
- **One-attempt-per-user** rule enforced on the frontend.
- Error handling for: failed quiz load, failed answer save, failed manual submit, **auto-submit edge cases**.

BACKEND REQUIREMENTS

5.A Start Quiz

Start Quiz Endpoint [**POST** /candidate/quizzes/:id/start]

- Return questions **only when**: (a) quiz in live window, (b) candidate is assigned, (c) **not already submitted**.
- Calling start **after submission** must return **409 Conflict**, never new questions.

Hide Correct Answers

- Question payload must **not** include **correctOptionIds / isCorrect** field.
- Strip the answer key **at the serialization layer** -- never rely on the client to ignore it.
- Verify with a **network inspector** before shipping -- leaked correct answers fail this entire milestone.

5.B End Quiz / Submission

End Quiz Endpoint [**POST** /candidate/quizzes/:id/submit]

- Persist candidate's answers and **actual submission timestamp** and the **start timestamp**.
- **Time guard**: submissions after startedAt + duration treated as **auto-submitted**, capped at deadline.
- **Idempotent** -- re-calling submit returns the existing attempt; **no double-scoring**, no data loss.
- Reject submissions for quizzes not in a submittable state with **409**.

5.C Score Calculation

Store Score on Submission

- Calculate score **once**, at submission time, and **persist** it on the attempt.
- All later reads (results, analytics) use the **stored value** -- never re-derive on read.
- Results are **deterministic** even if a question is later edited or deleted.

Scoring Correctness

- **Single-choice / True-False**: full marks if option matches; **zero** otherwise.
- **Multi-select**: full marks **only** when selected set **exactly matches** correct set -- no extras, no missing.
- **Unanswered** questions score **zero** (never negative unless explicitly specified).
- Handle all edge cases: empty submission, all-wrong, mixed types, single multi-select question.

5.D Attempt Integrity

One Attempt Per Candidate

- Candidate submits **exactly one** attempt per quiz.
- **Concurrent submits** (two browser tabs) must converge to **one stored attempt**.

RULES & CONSTRAINTS

- Correct answers **never** leave the server until the result phase (M6).
- Score computed **at submit** and persisted -- reads are **deterministic**.
- Submit is **idempotent** and capped at deadline; auto-submit client-trusted.
- One attempt per candidate per quiz, guaranteed by a **unique index**.

Milestone 6: Results and Analytics Flow

Result experience for all three roles. Score is calculated immediately after submission. Frontend: pixel-perfect result screens for Admin, Instructor, and User. Backend: result numbers must be deterministic -- the same request returns the same numbers every call.

FRONTEND REQUIREMENTS

UI -- Pixel Perfect (All 3 Roles)

- **Pixel-perfect** result UI for all 3 roles -- **Admin, Instructor, and User**.
- Modal/screen layouts match design expectations across all 3 roles.

Admin + Instructor View Result Flow

- View Result in both **Admin** and **Instructor** flows.
- Result view includes: **pie chart, stats cards, result table**; integrated via single API + Redux.
- **Instructor** result flow includes a working **View Details** action button.
- Search bar within the result table via API + Redux.
- Proper **empty states** for result table including **search empty state**.

Instructor Analytics Page

- **Analytics page** shows all completed quizzes with **backend pagination + Redux**.
- Search via **API + Redux + debounce/throttling**.
- Empty states for: **quiz list empty** and **search empty**.

Instructor Detail Result

- Clicking **View Details** opens **Instructor Detail Result modal** matching design.

- **Separate API call** on View Details click, integrated via Redux.

User Result View

- **View Result** opens a **score modal** with result summary matching design expectations.
- User can also open a **detailed result view**; correct API called, preferably via Redux.

BACKEND REQUIREMENTS

6.A Result Views

Candidate Result View [GET /candidate/quizzes/:id/result]

- Return own attempt: **total score**, **per-question breakdown** (selected vs correct), **time taken**.
- Only available **after submission**; before that return **404** or clear 'not submitted' payload.
- This is the **only** candidate endpoint that may include **correct answers**.

Instructor Aggregated View [GET /instructor/quizzes/:id/results]

- Aggregated metrics: total participants, total submitted, **average score**, highest/lowest, **score distribution**.
- Paginated per-candidate list (name, score, time taken); search by candidate name.
- **Ownership guard** -- only the assigned instructor sees this.

Admin Platform Summary [GET /admin/analytics]

- Cross-quiz summary: total quizzes by status, total candidates/instructors, **average attempt rate**.
- Optional filters by **date range** and **instructor**. **Admin-only** access.

6.B Determinism & Consistency

- **Same request = same numbers** across multiple calls -- no recomputation drift.
- Aggregates derive from the **stored score** on each attempt, **never** from re-grading.
- Sort orders must be **stable** (deterministic tiebreakers).

6.C Access Control -- Result Visibility

- Candidate endpoints **leak nothing** about other candidates' answers or scores.
- Instructor result endpoints scoped to **quizzes that instructor owns**.
- Direct ID probing must return **403/404**, never partial data.

RULES & CONSTRAINTS

- Result numbers derived from **stored scores** -- reads are pure.
- Correct answers revealed **only** via the candidate's own result endpoint, **after submission**.
- **Cross-tenant leakage** (instructor -> other instructor, candidate -> other candidate) is **forbidden**.

Milestone 7: Brownie Points / Advanced Completion

Advanced improvements beyond the core product. Frontend: dark theme with a token system and full mobile responsiveness. Backend: comprehensive test coverage, a README that lets a new dev clone and run in five minutes, and a Swagger spec that acts as the FE integration contract.

FRONTEND REQUIREMENTS

- **Dark theme** implemented across the application using a proper **token system**.
- **Mobile responsive** screens across all 6 milestones, **pixel perfect** as per design.
- Frontend **test coverage > 90%** with proper **unit and integration** test coverage for core modules.

BACKEND REQUIREMENTS

7.A Testing

Unit Test Coverage

- Aim for **>= 80%** coverage on backend logic -- controllers, services, validators, scoring.
- Cover **happy paths**, **edge cases** (empty inputs, boundary timestamps), and **error paths** (401, 403, 409).
- Configure a coverage report (**Jest --coverage**) and add a script to **package.json**.

Test Quality & Hermeticity

- Tests are **hermetic** -- no real DB, no real network, no real clock.
- Use **in-memory MongoDB** (e.g. mongodb-memory-server) or **mocks** for the data layer.
- Tests are **deterministic** -- same input, same result, every run, every order.
- **No setTimeout sleeps**; mock timers when testing the examination engine.

7.B Documentation

README

- Project overview, tech stack, folder structure.
- Prerequisites (Node version, MongoDB), **environment variables (.env.example)**, setup commands.
- How to run dev server, run tests, view coverage; links to **Swagger** and design notes.

API & Design Notes

- Architecture note: module boundaries, request flow, data model summary.
- Decisions worth recording: **why JWT-in-cookie**, **why score-on-submit**, **why soft-delete**.

Swagger / OpenAPI Spec [\[GET /api-docs\]](#)

- Document **every endpoint** -- method, path, request schema, response schema, error responses.
- Serve **Swagger UI** on a local route so the FE team can explore interactively.
- Keep spec **in sync with the code** -- out-of-date docs are worse than no docs.

RULES & CONSTRAINTS

- Tests must run **offline** -- no real DB, no real network.
- Coverage report must be runnable with a **single npm script**.
- **OpenAPI spec is the contract** -- the FE team should never have to read your controllers.

Build Order & Submission

Build milestones in order. M2-M6 depend on M1's auth layer. M5 depends on the quiz model from M2/M3 and the assignment guard from M4. M6 depends on the stored score from M5. Skipping ahead almost always means rewriting later.

Definition of Done -- Per Milestone

- All endpoints return correct payloads for **happy + edge + error** cases.
- All **state guards**, **role guards**, and **ownership guards** are wired in.
- Endpoints **documented in Swagger** and **covered by tests**.
- Frontend can hit endpoints directly **without a single ad-hoc patch**.

What Graders Will Probe

- **Auth boundaries** -- does an unauthenticated / wrong-role request get the right **401 / 403**?
- **State lifecycles** -- can you cancel a completed quiz? (you should **not** be able to)
- **Idempotency** -- what happens when you call submit twice? logout twice?

- **Determinism** -- same result endpoint, called five times, returns the **exact same numbers**?
- **Leakage** -- do questions, answers, or cross-tenant data slip through anywhere?

Final Word

- Build small, ship often -- **commit per feature**, not per milestone.
- Write the test alongside the controller -- it **doubles as your spec**.
- When in doubt, prefer the **safer behaviour** -- 403 over 200, hash over store, persist over recompute.
- Have fun. QuizArena is a **real-shaped product** -- build it like one.